Conda: A Working Manual

Environments for genomics on your Mac and on Talapas

Prepared for Bill Cresko

2026-11-02

Table of contents

1 The big picture 2
- 1.1 Your setup at a glance . . . 2

2 The environment lifecycle 3
- 2.1 Create . . . 3
- 2.2 Activate and deactivate . . . 3
- 2.3 Install into it . . . 3
- 2.4 Inspect . . . 4
- 2.5 Remove . . . 4

3 Quick reference card 4

4 Conda for genomics work 4
- 4.1 Why conda won in bioinformatics . . . 4
- 4.2 Patterns that work well . . . 5
- 4.3 Speed note: the solver . . . 5

5 Reproducibility: the environment.yml file 5
- 5.1 Export an environment you built by hand . . . 6
- 5.2 A hand-written recipe . . . 6
- 5.3 Rebuild anywhere . . . 6

6 Conda on Talapas 6
- 6.1 Loading conda . . . 6
- 6.2 Creating environments on the cluster . . . 7
- 6.3 Using an environment in a SLURM batch job . . . 7
- 6.4 The laptop–cluster loop . . . 8

7 Environments in Positron and notebooks 8

8 The wider ecosystem: conda alternatives and relatives 8
- 8.1 At a glance . . . 8
- 8.2 venv: Python's built-in environments . . . 9

8.3 mamba and micromamba: conda, faster 10
8.4 uv: the new Python-world standard 10
8.5 pixi: conda, reimagined per-project 11
8.6 And for R: renv 11

9 Troubleshooting **11**

10 Glossary **12**

11 Further reading **14**

1 The big picture

Conda is a **package and environment manager**: a tool that installs software into self-contained folders called *environments* and lets you switch between them. Each environment contains its own Python (or R), its own libraries, and — crucially for genomics — its own compiled command-line tools such as `samtools` or `bwa`. Nothing inside an environment can conflict with anything outside it, and deleting an environment removes it completely.

Two ideas explain almost everything about how conda behaves:

1. **An environment is a folder.** Creating one builds a folder (on your Mac, under `~/miniforge3/envs/`); activating one simply puts that folder first on your shell's PATH so its copies of `python`, `samtools`, etc. answer first; deactivating removes it again. There is no magic beyond PATH manipulation.
2. **Packages come from channels.** A channel is an online repository of pre-built software. You have two configured: **conda-forge** (the community channel for general scientific software) and **bioconda** (the bioinformatics channel, built on top of conda-forge). Between them, nearly every tool in a genomics pipeline is one `conda install` away — no compiling from source.

1.1 Your setup at a glance

As configured on your MacBook Pro (October 2026):

Component	Location	Role
Python 3.14 (python.org)	`/Library/Frameworks/Python.framework/Versions/3.14/`	Everyday Python for scripts, notebooks, teaching
Miniforge (conda)	`~/miniforge3/`	Creates environments on demand; otherwise stays out of the way
Apple's system Python 3.9	`/usr/bin/python3`	Belongs to macOS; leave it alone

Key configuration choices already made: `auto_activate` is **off** (conda never hijacks a new terminal; your prompt shows `(name)` only when you explicitly activate), channels are set to **conda-forge + bioconda** with **strict priority** (prevents mixed-channel version conflicts), and conda is initialized for **zsh** (your actual shell).

> **The one rule that prevents most confusion**
>
> Every "module not found" or "command not found" mystery is really the question *which Python/tool answered the phone?* Check with `which python3` or `which samtools`. If the path surprises you, look at your prompt: is an environment active that shouldn't be, or inactive that should be?

2 The environment lifecycle

2.1 Create

Name the environment after the project and declare what goes in it:

```
conda create -n neuromast python=3.12 numpy pandas samtools
```

Conda solves all version compatibility among the requested packages and builds the environment under `~/miniforge3/envs/neuromast`. You can add packages later, so the initial list need not be complete — but pinning the Python version (`python=3.12`) from the start is good practice for reproducibility.

2.2 Activate and deactivate

```
conda activate neuromast     # prompt now shows (neuromast)
# ... work ...
conda deactivate             # back to normal; 3.14 in charge again
```

Activation is **per terminal window**, not global. A second window starts in your normal shell state. The environment keeps everything between sessions; activation just switches which toolbox is open.

2.3 Install into it

With the environment active:

```
conda install bwa bcftools        # preferred: conda first
pip install some-rare-package     # fallback: pip works inside an active env
```

The rule of thumb is **conda first, pip for stragglers, in that order**. Installing conda packages *after* pip packages in the same environment occasionally tangles dependencies; if an environment gets confused, the clean fix is to delete and recreate it (seconds of your attention, minutes of compute).

2.4 Inspect

```
conda env list     # all environments; * marks the active one
conda list         # packages inside the active environment
conda list samtools  # is this one package installed, and which version?
```

2.5 Remove

```
conda env remove -n neuromast
```

Environments are disposable by design. When in doubt, recreate rather than repair.

3 Quick reference card

Task	Command
Create environment	`conda create -n NAME python=3.12 pkg1 pkg2`
Create from a recipe file	`conda env create -f environment.yml`
Activate / deactivate	`conda activate NAME` / `conda deactivate`
Install a package	`conda install PKG` (env active)
Search for a package	`conda search PKG`
List environments	`conda env list`
List packages in active env	`conda list`
Export recipe	`conda env export --from-history > environment.yml`
Remove environment	`conda env remove -n NAME`
Update one package	`conda update PKG`
Free disk from caches	`conda clean --all`
Which tool is answering?	`which python3` / `which TOOL`

4 Conda for genomics work

4.1 Why conda won in bioinformatics

Genomics tools are notoriously painful to compile — C/C++ codebases with chains of library dependencies. The **bioconda** channel distributes thousands of them pre-built: `samtools`, `bcftools`, `bwa`, `bowtie2`, `vcftools`, `plink`, `angsd`, `gatk4`, `stacks`, and far more. A pipeline's whole toolchain becomes one command:

```
conda create -n radseq-2026 python=3.12 stacks samtools bwa bcftools vcftools
conda activate radseq-2026
```

That environment is also a *record*: six months later, `conda list` tells you exactly which version of every tool produced your results — the provenance a methods section needs.

4.2 Patterns that work well

One environment per project, named for the project. Not one giant environment for everything — the more packages share an environment, the harder conda's version-solving becomes, and the more likely an update for one tool breaks another.

A separate environment for a troublesome tool. Some tools pin old Python versions or conflict with everything (older ML tools are notorious). Give such a tool its own environment and the problem disappears:

```
conda create -n gatk-only gatk4
```

Pin versions for production pipelines. `conda install stacks=2.68` rather than whatever is newest, once a pipeline is producing results you'll publish.

Python single-cell stack. If you ever want the Python side of single-cell analysis alongside your R/Seurat workflow, it lives happily in an environment: `conda create -n sc-python python=3.12 scanpy anndata leidenalg`.

> **i** R inside conda?
>
> Conda *can* manage R and many R packages (`r-base`, `r-tidyverse`, bioconductor packages as `bioconductor-*`). On a laptop where you already maintain R and Seurat directly, there is usually no reason to move; conda-R matters mostly on clusters where you cannot install R system-wide, or when a pipeline needs a pinned, reproducible R version.

4.3 Speed note: the solver

Modern conda uses the fast **libmamba** solver by default, so the old advice to install "mamba" as a faster replacement is mostly obsolete. If a `conda create` on an older conda (such as a cluster module) grinds for minutes, add `--solver=libmamba` — Talapas's own documentation recommends exactly this flag.

5 Reproducibility: the environment.yml file

This is the payoff habit. An `environment.yml` is a small text file describing an environment — name, channels, packages — that anyone (including future you, or a cluster) can rebuild from.

5.1 Export an environment you built by hand

With the environment active:

```
conda env export --from-history | sed '/prefix:/d' > environment.yml
```

`--from-history` records only what you *asked for*, not all ~200 resolved dependencies — which keeps the file short and portable across operating systems (a full export from a Mac often fails to rebuild on Linux because low-level dependencies differ). The `sed` removes the machine-specific `prefix:` line; this is the exact form Talapas's documentation recommends for sharing.

5.2 A hand-written recipe

You can equally write one from scratch:

```
name: radseq-2026
channels:
  - bioconda
  - conda-forge
dependencies:
  - python=3.12
  - stacks=2.68
  - samtools
  - bwa
  - bcftools
```

5.3 Rebuild anywhere

```
conda env create -f environment.yml
```

The habit: keep an `environment.yml` in each project's repository, next to the code. It is simultaneously your collaborator instructions, your Talapas deployment recipe, and the environment section of your methods.

6 Conda on Talapas

Talapas 2 provides conda as a **module** — you load it rather than install it. Your laptop Miniforge and the cluster conda behave identically once loaded; the `environment.yml` file is the bridge between them.

6.1 Loading conda

```
ssh <duckid>@login.talapas.uoregon.edu
module load miniconda3/20240410
```

(`module spider miniconda` shows whether a newer module version exists; `module list` shows what you have loaded.)

6.2 Creating environments on the cluster

Exactly as on your laptop, with the recommended solver flag:

```
conda env create --solver=libmamba -f environment.yml
# or manually:
conda create --solver=libmamba --name radseq-2026 python=3.12 stacks samtools
```

RACS recommends adding a **date tag** to environment names on the cluster — `radseq-20261001` — so successive rebuilds of a pipeline's environment stay distinguishable.

> ⚠ Watch your home-directory quota
>
> Conda environments are large (often several GB) and by default land in your home directory, which on clusters has a limited quota. If environments for a project should live in (or be shared through) your PIRG's project space instead, `conda create --prefix /path/to/project/envs/NAME` places one at an explicit path — check current RACS storage guidance for where your group's space is and what it allows. `conda clean --all` reclaims cached downloads anywhere.

6.3 Using an environment in a SLURM batch job

Login nodes are for editing and submitting, not computing. A batch script loads the module and activates the environment before running the pipeline:

```
#!/bin/bash
#SBATCH --account=<yourpirg>
#SBATCH --partition=compute
#SBATCH --job-name=stacks_run
#SBATCH --output=stacks_%j.out
#SBATCH --error=stacks_%j.err
#SBATCH --time=0-08:00:00
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G

module load miniconda3/20240410
```

```
conda activate radseq-20261001

process_radtags -p ./raw -o ./samples ...
```

Submit with `sbatch script.sh`; check with `squeue --me`. Jobs longer than a day go to the `computelong` partition; high-memory and GPU partitions also exist.

6.4 The laptop–cluster loop

The complete round trip, which is the real reason to learn all of this:

1. Develop and test an analysis in a small environment on your Mac.
2. `conda env export --from-history | sed '/prefix:/d' > environment.yml`; commit it to the project repo.
3. On Talapas: `module load miniconda3/20240410`, then `conda env create --solver=libmamba -f environment.yml`.
4. Run at scale via SLURM, with the identical toolchain.

7 Environments in Positron and notebooks

Positron's interpreter picker (top-right corner) lists conda environments automatically alongside your main 3.14, labeled like "Python 3.12 (neuromast)". Selecting one runs your notebook inside it — no terminal activation needed. Two notes:

- A notebook kernel requires **ipykernel** in the environment. Positron usually offers to install it when you first select the environment; or include it at creation time (`conda create -n NAME python=3.12 ipykernel ...`).
- If a freshly created environment doesn't appear in the picker, restart Positron so it rescans.

And the habit that keeps packages landing in the right place regardless of which interpreter a notebook uses: install from *inside the notebook* with a `%pip install <package>` cell.

8 The wider ecosystem: conda alternatives and relatives

Conda is not the only environment framework, and you will meet the others constantly — in tutorials, in collaborators' repositories, and in cluster documentation. They fall into two families: tools that manage **Python packages only** (venv, pip, uv), and tools that manage the **full conda ecosystem** of compiled software (mamba, micromamba, pixi). Knowing which family a tool belongs to tells you most of what you need.

8.1 At a glance

Tool	What it manages	Installs from	Compiled tools (samtools etc.)?	One-line verdict
conda	Python/R + any compiled software	conda channels (conda-forge, bioconda)	**Yes**	Your default for genomics
mamba	Same as conda (drop-in replacement)	conda channels	**Yes**	Faster conda; mostly obsolete now that conda uses the libmamba solver
micromamba	Same, as a single tiny binary	conda channels	**Yes**	Conda for containers and CI, no installation ceremony
pixi	Conda packages, per *project* rather than per named env	conda channels + PyPI	**Yes**	The modern project-first take on conda; worth watching
venv	Python packages only	PyPI (via pip)	No	Built into Python; fine for pure-Python work
uv	Python packages, Python versions, projects	PyPI	No	Extremely fast modern replacement for venv + pip
renv	R packages, per project	CRAN/Bioconductor	No	The R-world analogue; complements rather than competes

The dividing line in the fourth column is the practical one: if a workflow needs `samtools`, `bwa`, or anything else from bioconda, only the conda-family tools can provide it. No amount of `pip install` produces a working `bwa`.

8.2 venv: Python's built-in environments

`venv` ships with Python itself and is the standard tool for pure-Python projects:

```
python3 -m venv .venv              # create (conventionally hidden, inside the project)
source .venv/bin/activate          # activate; prompt shows (.venv)
pip install numpy pandas           # install from PyPI
deactivate
```

Three differences from conda worth internalizing:

1. **A venv borrows an existing Python** rather than containing its own — so it is small and fast to create, but every venv on your machine depends on the interpreter it was created from.
2. **It lives inside the project folder** (`.venv/`), not in a central registry. There is no `venv env list`; the environment *is* the directory.
3. **Its recipe is `requirements.txt`** (or `pyproject.toml`), the pip-world counterpart of `environment.yml`: `pip freeze > requirements.txt` to export, `pip install -r requirements.txt` to rebuild.

When a Python tutorial says "create a virtual environment," this is what it means. (You may also see `virtualenv`, the older third-party package venv was distilled from — for everyday purposes they are the same thing.)

8.3 mamba and micromamba: conda, faster

`mamba` began as a drop-in reimplementation of conda in C++, built because conda's original dependency solver could grind for many minutes. Every command is identical — `mamba create`, `mamba install`, `mamba activate` — pointed at the same channels and producing the same environments. It was so successful that conda absorbed its solver: since conda 23.10, **libmamba is conda's default solver**, which is why plain `conda` on your Mac is fast and why the main reason to install mamba separately has evaporated.

You will still meet mamba in two places:

- **Older cluster installations** whose conda predates the merger. There, `mamba create ...` (if installed) or `conda create --solver=libmamba ...` (the flag Talapas documentation recommends) restores modern speed.
- **micromamba**, a single self-contained binary needing no base environment and no installer. It is the standard way to get conda environments inside Docker/Apptainer containers and CI pipelines, where installing full Miniforge would be absurd. If you ever containerize a pipeline, micromamba is the tool you'll reach for.

8.4 uv: the new Python-world standard

`uv` (from Astral, the makers of the `ruff` linter) is a Rust reimplementation of the whole venv + pip toolchain, routinely 10–100× faster, and it has swept through the Python community since 2024. It adds real project management on top: `uv init` starts a project, `uv add pandas` records the dependency in `pyproject.toml` and updates a lockfile, `uv run script.py` executes in the project's environment — creating it on the fly if needed. It will even download and manage Python interpreters for you.

The limitation is the family line: uv installs **from PyPI only**. For a pure-Python package, a teaching example, or a quick script, it is wonderful. For a genomics pipeline needing bioconda's compiled tools, it cannot help. The pragmatic division of labor:

- **conda** for anything touching bioinformatics tools or heading to Talapas
- **uv** (or plain venv) for pure-Python side projects, if you want the modern tooling

8.5 pixi: conda, reimagined per-project

`pixi` (from prefix.dev, by core mamba developers) is the newest member of the conda family and the one most worth watching for scientific work. It installs the *same packages from the same channels* — conda-forge, bioconda — but organizes them the way venv and uv do: **per project, not per named central environment**. A `pixi.toml` plus an automatically maintained lockfile live in the project repository; `pixi run` executes commands in the project's environment without any activate step; collaborators get a bit-for-bit identical environment from the lockfile.

In effect, pixi is `environment.yml` with the reproducibility dialed up (lockfiles pin exact versions, where `--from-history` exports record only your requests) and the environment bookkeeping made automatic. It is increasingly common in bioinformatics repositories. Nothing about your conda setup needs to change today — but when you see a repo with a `pixi.toml`, that is what you're looking at, and `pixi install` in that directory rebuilds the authors' exact environment.

8.6 And for R: renv

On the R side, the analogous per-project tool is `renv`: `renv::init()` gives a project its own package library, `renv.lock` records exact versions, `renv::restore()` rebuilds them. It manages R packages only (conda can manage R itself; renv assumes R is already there), so the two compose naturally: conda provides a pinned R on a cluster, renv pins the packages inside it.

> So what should *you* use?
>
> Nothing needs to change. Conda via Miniforge remains the right default for your work: it is the only family that delivers bioconda's compiled tools, it is what Talapas provides, and the `environment.yml` loop runs through it. File the rest as vocabulary: venv/uv when a pure-Python project or tutorial calls for them, micromamba if you containerize, pixi as the likely future shape of conda-style work.

9 Troubleshooting

`zsh: command not found: conda` — conda's startup block isn't wired into the shell you're using. Fix: `~/miniforge3/bin/conda init zsh`, then open a new terminal. (This bit you twice: both the old Anaconda and the first Miniforge attempt initialized bash instead of zsh.)

`(base)` appears in every new terminal — auto-activation got turned back on. Fix: `conda config --set auto_activate false`, new terminal.

`ModuleNotFoundError` in a notebook — the package isn't installed in the interpreter the notebook is using (it may be installed somewhere else). Fix: run `%pip install <package>` in a notebook cell, or check the kernel in the notebook toolbar matches the environment you intended.

Wrong tool answering — `which python3` or `which samtools` shows an unexpected path. Deactivate stray environments (`conda deactivate`), or activate the one you meant.

pip-installed package "missing" its dependencies — a package installed while an IDE's own tooling was visible can skip dependencies. Fix: `python3 -m pip install --force-reinstall <package>` from a plain terminal.

Conda solve is painfully slow — add `--solver=libmamba` (older conda versions), reduce the environment's size, or split troublesome tools into their own environment.

Disk filling up — `conda clean --all` clears package caches; `conda env list` then `conda env remove -n NAME` retires environments you no longer use.

An environment is just broken — don't debug it. `conda env remove -n NAME` and recreate from the `environment.yml`. This is the designed workflow, not an admission of defeat.

10 Glossary

Anaconda — A company, and its large (~5 GB) conda distribution bundling hundreds of packages you may never use. You removed one; Miniforge is the lean modern alternative.

activate / deactivate — Switching an environment's folder onto / off of your shell's PATH, so its tools answer first (or stop answering).

base — The environment conda itself lives in. Best left nearly empty; create named environments for actual work.

bioconda — The community channel distributing pre-built bioinformatics software (samtools, Stacks, GATK, and thousands more). Built on top of conda-forge.

channel — An online repository of conda packages. Yours: conda-forge and bioconda, with strict priority (packages resolve from one channel consistently rather than mixing).

conda — The package/environment manager itself; also the command.

conda-forge — The large community-maintained channel for general scientific software; Miniforge's default.

dependency — A package some other package needs to run. Conda's core job is *solving* dependencies: finding versions of everything that coexist.

environment — A self-contained folder holding a Python/R plus packages and tools, switched on and off by activation. The unit of isolation and reproducibility.

environment.yml — The small text recipe (name, channels, packages) an environment can be rebuilt from. One per project, kept in the repo.

HPC — High-performance computing; shared clusters like Talapas.

ipykernel — The package that lets a Python environment serve as a notebook kernel in Positron/Jupyter.

kernel — The running Python process behind a notebook; restarting it clears memory and picks up newly installed packages.

libmamba / mamba — A fast dependency solver, now conda's default; `mamba` was a drop-in faster conda from before this merger, still seen on clusters.

micromamba — mamba as a single self-contained binary with no base environment; the standard way to use conda packages inside containers and CI pipelines.

Miniconda — Anaconda's minimal installer (conda + base Python only); defaults to Anaconda's commercial channel.

Miniforge — The community equivalent of Miniconda, defaulting to conda-forge. What's on your Mac, at `~/miniforge3`.

module (Lmod) — The cluster mechanism for making software available per-session: `module load miniconda3/20240410`. Unrelated to Python modules.

PATH — The ordered list of folders your shell searches for commands. Which Python answers `python3` is determined entirely by PATH order; activation works by editing it.

PIRG — Principal Investigator Research Group: the account/storage unit on Talapas; your `#SBATCH --account` value.

pixi — A project-centric package manager for the conda ecosystem: same channels (conda-forge, bioconda), but environments defined per repository by a `pixi.toml` and exact lockfile rather than by named central environments.

pip — Python's own package installer, pulling from PyPI. Works inside conda environments; use after conda packages, not before.

PyPI — The Python Package Index, pip's repository. Python-only, unlike conda channels, which also carry compiled tools.

SLURM — Talapas's job scheduler. `sbatch` submits a script; `squeue --me` shows your jobs; partitions (`compute`, `computelong`) are queues with different limits.

solver — The component that picks mutually compatible package versions when creating or updating an environment.

renv — R's per-project package manager: a project-local library plus an `renv.lock` of exact versions. Manages R packages only; composes with conda rather than competing.

requirements.txt — The pip-world recipe file (a plain list of PyPI packages); the counterpart of `environment.yml` for venv/uv environments.

strict channel priority — A conda setting that resolves each package from the highest-priority channel that has it, preventing subtle mixed-channel breakage.

uv — A very fast Rust replacement for venv + pip with built-in project management (`pyproject.toml` + lockfile) and Python version management. PyPI only: no compiled bioinformatics tools.

venv — Python's built-in lightweight environment tool: borrows an existing Python interpreter, manages Python packages only, lives inside the project folder as `.venv/`. Conda environments additionally contain their own Python and non-Python tools.

virtualenv — The older third-party package that venv was distilled from; for everyday purposes, the same thing as venv.

YAML — The indentation-based text format of `environment.yml` (and Quarto headers).

11 Further reading

- Talapas 2 Knowledge Base: https://uoracs.github.io/talapas2-knowledge-base/
- Creating personal conda environments on Talapas: https://uoracs.github.io/talapas2-knowledge-base/docs/how-to_articles/how-to_create_personal_conda_envs/
- Sharing/exporting conda environments on Talapas: https://uoracs.github.io/talapas2-knowledge-base/docs/how-to_articles/how-to_share_conda_envs/
- Bioconda package index: https://bioconda.github.io/
- Conda documentation: https://docs.conda.io/
- Miniforge: https://github.com/conda-forge/miniforge
- mamba / micromamba documentation: https://mamba.readthedocs.io/
- uv documentation: https://docs.astral.sh/uv/
- pixi documentation: https://pixi.sh/
- venv (Python standard library): https://docs.python.org/3/library/venv.html
- renv: https://rstudio.github.io/renv/